```
}
```

You may use the **`assertMissing`** function offered by the **`Storage`** facade to assert that a certain file does not exist:

```
Storage::fake('cats');
// Application Logic
Storage::disk('cats')->assertMissing('missing.jpg');
```

To properly test your application's validation requirements, you may provide the width, height, and size of the picture (in kilobytes) when generating files using the **`fake`** method offered by the **`UploadedFile`** class:

```
UploadedFile::fake()->image('cat.jpg', $width, $height)-
>size(100);
```

The **`create`** method may be used to produce any other sort of file in addition to images:

```
UploadedFile::fake()->create('document.pdf', $sizeInKilobytes);
```

If necessary, you may specify the MIME type that the file should return directly by passing a **`$mimeType`** parameter to the method:

```
UploadedFile::fake()->create(
  'document.pdf', $sizeInKilobytes, 'application/pdf'
);
```

Debugging in Laravel

Using Xdebug for step debugging is supported by Homestead. For instance, PHP will establish a connection with your IDE when you open a page in your browser, enabling you to view and edit the code that is now running.

Xdebug is launched and ready to receive connections by default. Use the **`sudo phpenmod xdebug`** command in your Homestead virtual machine if you need to activate Xdebug on the CLI. Now enable debugging by following the instructions in your IDE. Lastly, set up your browser such that a bookmarklet or extension will launch Xdebug.

It is easier to autostart debugging functional tests that send queries to the web server than it is to change the tests such that debugging is triggered by passing via a special header or cookie. In your Homestead virtual machine, edit the **`/etc/php/7.x/fpm/conf.d/20-xdebug.ini`** file and add the following settings to have Xdebug launch automatically: